
Микросервис уведомлений (Notification Service)

Цель этого задания — посмотреть, как ты проектируешь распределенные системы, работаешь с асинхронными процессами и обеспечиваешь качество кода через автоматизированное тестирование.

⚙️ Функциональные требования (Бизнес-требования)

- **Массовая рассылка уведомлений:** Система должна предоставлять API для запуска массовой отправки SMS или Email-сообщений. Инициатор запроса передает канал связи, текст сообщения и массив идентификаторов получателей.
- **Приоритезация трафика:** Система должна гарантировать доставку критичных сообщений без задержек. Транзакционные уведомления (коды доступа, срочные изменения маршрутов) должны получать наивысший приоритет и отправляться вне очереди, обгоняя маркетинговые рассылки.
- **Детализация статусов доставки:** Система должна предоставлять API для запроса истории и текущего статуса всех уведомлений конкретного подписчика. Статусы:
 - в очереди (сообщение принято и ожидает отправки);
 - отправлено (передано шлюзу/провайдеру);
 - доставлено (подтверждено провайдером);
 - отброшено (ошибка доставки, несуществующий номер/email и т.д.).

☹️ Нефункциональные требования (Архитектура и качество)

- **Гарантия доставки сообщений (Reliability):**
 - **Персистентность:** Использование брокеров сообщений для хранения очереди.
 - **Модель доставки:** Поддержка семантики at-least-once. Огромным плюсом будет реализация exactly-once на уровне бизнес-логики.
 - **Retry-механизмы:** Автоматический повтор попыток отправки при временной недоступности шлюзов.
- **Дедубликация (Idempotency):** Защита от повторной отправки одного и того же сообщения при дублировании запросов от вызывающего сервиса.
- **Тестирование: Обязательное наличие интеграционных тестов** на основные сценарии. Мы хотим видеть автоматизированную проверку всей цепочки: от получения сообщения из очереди до корректного изменения статуса в базе данных и вызова нужного провайдера.
- **Cloud Native и развертывание:**

- Упаковка сервиса в Docker-образ.
- Весь проект (БД, брокер, кэш, приложение) должен запускаться одной командой `docker-compose up`.

✂ Рекомендуемый технологический стек

- Язык и фреймворк: **PHP (Laravel)** или **Python (FastAPI)**.
- База данных: **PostgreSQL**.
- Брокер сообщений: **Apache Kafka** или **RabbitMQ**.
- Кэш / In-memory хранилище: **Redis** (для дедубликации и контроля лимитов).

Примечание: для внешних шлюзов используй классы-заглушки (моки), которые имитируют работу реальных провайдеров.

✂ Как сдавать результат

1. Опубликуй код в публичный репозиторий на **GitLab** или **GitHub** и пришли ссылку.
2. В файле **README.md** опиши пошаговую инструкцию по запуску через `docker-compose`.
3. Предоставь описание API для тестирования. Ссылка на **Swagger** (OpenAPI) или приложенная Postman-коллекция будет идеальным решением.

Удачи! Мы ценим основательный подход к архитектуре и качеству кода.
Ждем твое решение!